부모용 진행 안내 · 구현과 검증

중3 정보·AI · 문제분해와코드 2차시 · 약 40분 · 수업 ID KR22-G09-INF-TESTCODE-L01

시작 전에

- 아이에게는 PPT를 **슬라이드 쇼**로 보여 주세요. 일반 보기 화면 아래쪽 노트에는 정답이 있습니다. 진행은 이 안내서를 보며 합니다.
- 준비물: 활동지, 연필, 지우개, 지난 차시 '문제 분해와 설계' 활동지(용돈 기록기 구조도·순서도), (선택) 블록 코딩 도구(엔트리, 스크래치 등)나 파이썬을 쓸 수 있는 컴퓨터 — 없으면 부모가 컴퓨터 역할을 합니다
- 정답을 바로 알려 주기보다 "어떻게 알았어?"라고 묻고 아이가 설명하게 하세요. 막히면 각 장의 '막히면'을 따르세요.
- 아이가 지치면 S09 뒤에서 나누어 이틀에 해도 됩니다.

오늘의 목표와 확인 방법

목표 A 설계한 기능을 하나씩 의사코드로 구현하고, 보통 값·경계 값·특별한 값을 넣은 테스트 사례표(입력·예상 결과·실제 결과·통과)를 만들어 예상 결과는 실행 전에 요구로 정한 뒤 코드를 따라가 결과를 검증한다.
확인 순서도를 의사코드로(S04), 테스트 사례표(S05), 어떤 사례를 넣을까요(S06), 알리기 기능 검증하기(S07), 혼자 해 보기 ①(S12), 마무리 확인(S14), 활동지 1·2·4
성공 기준 바뀐 요구(5000원 이하)에서 경계 값 사례 4999·5000·5001과 예상 결과를 바르게 쓰고, 처음 보는 반복 코드의 예상(3500)과 실제(1500) 결과를 견주어 통과 여부를 판단한다.

목표 B 통과하지 못한 사례의 입력으로 코드를 한 줄씩 따라가 예상과 달라지는 줄을 찾아 원인(조건)을 고치고, 모든 사례를 다시 실행해 검증한다.
확인 통과하지 못한 사례(S08), 어떻게 고칠까요(S09), 오류를 고치는 차례(S10), 도구로 실행해 확인해요(S11, 선택), 혼자 해 보기 ②③(S12), 활동지 2·3
성공 기준 사흘 합계 코드의 '날 < 3'을 '날 ≤ 3'으로 고치고, 오류가 드러나는 새 사례(셋째 날이 0이 아닌 값)를 만들어 다시 검증한다.

먼저 알고 있어야 할 것

- 문제 분해와 설계

S02에서 흔들리면 보충 장 S16부터 합니다. 보충으로도 어려우면 앞 차시(문제 분해와 설계)부터 다시 다지는 것이 좋습니다.

자주 나오는 오해

번호	아이가 이렇게 생각하면	이렇게 도와주세요
M1	프로그램을 끝까지 다 만든 뒤 한 번에 실행해 보면 된다고 생각한다.	기능 하나를 구현할 때마다 바로 검증하는 흐름을 따르고, 오류가 방금 만든 기능 안에 있다는 장점을 말한다(S03).
M2	실행해서 나온 결과를 보고 예상 결과를 적어, 틀린 결과도 통과로 표시한다.	예상 결과는 실행 전에 요구 문장으로 정한다(S05, S07, S13).
M3	보통 값 몇 개가 통과하면 프로그램이 맞다고 생각한다.	5000·1000은 통과하지만 경계 값 3000에서 오류가 드러나는 것을 직접 확인하고, 셋째 날이 0이면 오류가 숨는 사례를 본다(S06, S07, S12).
M4	틀린 사례 하나만 맞도록 덧붙여 고치거나, 고친 뒤 그 사례만 다시 확인한다.	원인(조건)을 고치고 모든 사례를 다시 실행한다(S09, S10).

진행 순서의 (M1), (M2) 같은 표시는 이 표의 번호입니다.

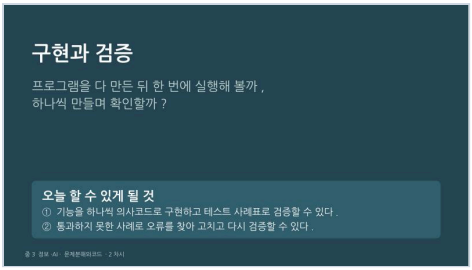
오늘 다루지 않는 것 앞 차시에서 설계한 용돈 기록기의 기능 셋(남은 돈 계산하기, 아껴 쓰라고 알리기, 사흘 쓴 돈 합계 내기)을 의사코드(입력·출력·대입·만약/아니면·인 동안 반복)로 구현하고, 테스트 사례표로 검증하며, 경계 값으로 드러난 조건 오류를 찾아 고쳐 모든 사례를 다시 실행하는 데까지 다룬다. 특정 프로그래밍 언어의 문법, 자동 테스트 도구, 예외 처리(글자를 잘못 넣은 경우), 효율 비교는 다루지 않는다. 도구로 구현하는 것은 선택이다. 금액과 기준(3000원·5000원)은 수업을 위해 정한 예시이며, S07·S12 코드의 오류는 일부러 넣은 것이다.

진행 순서

장	하는 일	시간	목표
S01	오늘의 질문과 목표	약 2분	A, B
S02	선수 확인(기능 구조도, 판단 순서도, 입력·처리·출력)	약 3분	—
S03	기능을 하나씩 구현하고 바로 검증하는 흐름	약 2분	A
S04	순서도를 의사코드로 옮기기(남은 돈 계산하기)	약 3분	A
S05	테스트 사례표 만들기(시범)	약 3분	A
S06	어떤 테스트 사례를 고를까(보통 값·경계 값·특별한 값)	약 2분	A
S07	테스트 사례표 채우고 코드 따라가기(도움 받아)	약 4분	A
S08	통과하지 못한 사례가 알려 주는 오류의 위치	약 3분	B
S09	오개념 비교(특정 사례만 맞추는 고치기 vs 원인 고치고 모두 다시 검증)	약 2분	B
S10	오류를 찾아 고치는 차례(디버깅)	약 2분	B
S11	도구로 구현하고 사례 실행(선택)	약 3분	A, B
S12	혼자 해 보기(사흘 합계 기능 검증과 디버깅)	약 5분	A, B

S13	잘한 모습과 다시 생각해 볼 모습	약 2분	A, B
S14	마무리 확인(바뀐 요구에 맞는 경계 값 사례 만들기)	약 3분	A
S15	정리와 다음 학습	약 2분	A, B
S16	보충(조건의 참·거짓 판단)	약 4분	—

장별 안내



S01 구현과 검증 · 오늘의 질문과 목표 · 약 2분

지난 차시에 설계한 용돈 기록기의 기능을 하나씩 의사코드로 구현하고, 테스트 사례표로 맞게 작동하는지 검증하며 오류를 고친다고 알립니다.

이렇게 말해 보세요

- “지난 시간에 용돈 기록기를 작은 기능으로 나누고 순서도로 그렸지? 이제 실제로 만들 차례야.”
- “다 만들고 나서 실행했는데 틀린 답이 나오면, 어디가 틀렸는지 찾기 쉬울까?”

예상 답

- “어디가 틀렸는지 모를 것 같아요. 하나씩 해요.”

주의

- 준비물: 활동지, 연필, 지우개, 지난 차시 활동지(용돈 기록기 구조도·순서도).
- 컴퓨터 없이 종이로 합니다. 엔트리·스크래치 같은 블록 코딩 도구나 파이썬을 쓸 수 있으면 S11에서 직접 실행해 봅니다(선택).



S02 지난 학습 기억하기 · 선수 확인(기능 구조도, 판단 순서도, 입력·처리·출력) · 약 3분

앞서 배운 ‘문제 분해와 설계’에서 그린 기능 구조도, 판단이 있는 순서도의 경계 값, 기능 설계 차례를 기억하는지 확인합니다.

이렇게 말해 보세요

- “①구조도 맨 위 아래에 무엇이 셋 있었지?”
- “②요구는 ‘3000원보다 적으면 아껴 써요’였어. 3000은 3000보다 적어?”

정답

- ① 기록하기, 계산하기, 알려 주기
- ② 괜찮아요 (3000 < 3000은 아니요)
- ③ 입력, 처리(계산·판단·반복), 출력

막히면

- 두 가지 이상 어려우면 S16(보충)을 먼저 하고 S03으로 이어 갑니다.



S03 **하나씩 만들고 바로 확인** · 기능을 하나씩 구현하고 바로 검증하는 흐름 · 약 2분

구현(설계를 실행되는 프로그램으로 만드는 것)과 검증(요구대로 작동하는지 확인하는 것)의 뜻을 알려 주고, 기능 하나를 구현할 때마다 바로 검증하는 흐름을 보여 줍니다(M1).

이렇게 말해 보세요

- “구조도의 작은 기능 하나를 골라 코드로 만들고, 바로 맞는지 확인해. 맞으면 다음 기능으로 가.”
- “이렇게 하면 오류가 나도 방금 만든 기능 안에 있으니 찾기 쉬워.”

예상 답

- “레고도 한 칸씩 맞춰 보면서 만들어요.”

막히면

- 라면 끓이기에서 물이 끓는지 확인하고 면을 넣는 것처럼, 단계마다 확인한다고 비유합니다.

주의

- 실제 개발에서도 기능별로 작게 시험하는 것(단위 테스트)이 기본입니다.



S04 **순서도를 의사코드로** · 순서도를 의사코드로 옮기기(남은 돈 계산하기) · 약 3분

지난 차시에 그린 ‘남은 돈 계산하기’ 순서도의 상자 하나가 의사코드 한 줄이 된다는 것을 보여 줍니다. 의사코드는 특정 언어가 아니라 사람이 읽기 쉽게 쓴 코드입니다.

이렇게 말해 보세요

- “순서도의 입력 상자 두 개가 1, 2줄이 됐어. 처리 상자는 몇 줄일까?”
- “남은돈 5000, 쓴돈 1500을 넣고 한 줄씩 따라가 볼래? 3줄이 끝나면 남은돈은?”

예상 답

- “3줄이요.” / “3500이요.”

정답

- 남은돈 5000, 쓴돈 1500 → 3줄 뒤 남은돈 3500 → 출력 3500

막히면

- 지난 차시 활동지의 순서도를 옆에 놓고 상자와 줄을 하나씩 손가락으로 짝지어 봅니다.

주의

- ‘←’는 오른쪽을 먼저 계산해 왼쪽 상자(변수)에 넣는다는 뜻입니다(중1 ‘변수와 입출력’).



S05 **테스트 사례표** · 테스트 사례표 만들기(시범) · 약 3분

테스트 사례(넣어 볼 입력과 나와야 할 결과), 예상 결과(실행 전에 요구로 정한 결과), 실제 결과(실행해서 나온 결과), 통과(둘이 같음)의 뜻을 표로 보여 줍니다.

이렇게 말해 보세요

- “예상 결과는 언제 정해야 할까? 실행해 보고 나서 정하면 어떻게 될까?”
- “사례 2처럼 가진 돈을 다 쓰는 때도 넣어 봤어. 왜 이런 사례도 넣었을까?”

예상 답

- “실행한 걸 보고 정하면 틀린 것도 맞다고 하게 돼요.” / “0이 제대로 나오는지 보려고요.”

정답

- 사례 1: 남은돈 5000, 쓴돈 1500 → 예상 3500, 실제 3500, 통과 ○
- 사례 2: 남은돈 2000, 쓴돈 2000 → 예상 0, 실제 0, 통과 ○
- 사례 3: 남은돈 3000, 쓴돈 0 → 예상 3000, 실제 3000, 통과 ○

막히면

- 사례 1만 함께 계산하고(5000 − 1500), 사례 2·3은 아이가 계산해 표와 맞춰 봅니다.

주의

- 예상 결과를 실행 뒤에 실제 결과를 보고 쓰면 검증이 되지 않습니다(M2).



S06 **어떤 사례를 넣을까?** · 어떤 테스트 사례를 고를까(보통 값·경계 값·특별한 값) · 약 2분

테스트 사례를 고르는 세 가지(보통 값, 경계 값, 특별한 값)를 알려 줍니다. 경계 값은 조건의 기준(3000)과 그 바로 옆 값(2999)입니다(M3).

이렇게 말해 보세요

- “보통 값 5000과 1000만 넣어 보면 충분할까?”
- “‘3000원보다 적으면’에서 결과가 바뀌는 곳은 어디야? 그 근처 값을 꼭 넣어 봐야 해.”

예상 답

- “3000이랑 2999요.”

막히면

- 수직선에 3000을 표시하고, 왼쪽은 ‘아껴 써요’, 3000과 오른쪽은 ‘괜찮아요’라고 색을 나눠 그려 봅니다.

주의

- 모든 값을 다 넣어 볼 수는 없습니다. 그래서 오류가 잘 드러나는 사례를 골라 넣는 것이 중요합니다.



S07 알리기 기능 검증하기 · 테스트 사례표 채우고 코드 따라가기(도움 받아) · 약 4분

도움을 받으며 '아껴 쓰라고 알리기'의 테스트 사례표를 채웁니다. 예상 결과는 요구 문장으로 먼저 정하고, 그다음 코드를 한 줄씩 따라가 실제 결과를 씁니다. 이 코드에는 일부러 넣은 오류가 하나 있습니다(아이에게 미리 말하지 않습니다).

이렇게 말해 보세요

- "먼저 코드는 보지 말고, 요구 문장만 보고 네 사례의 예상 결과를 써 봐."
- "이제 코드를 따라가. 3000을 넣으면 2줄의 '3000 ≤ 3000'은 참이야, 거짓이야?"

정답

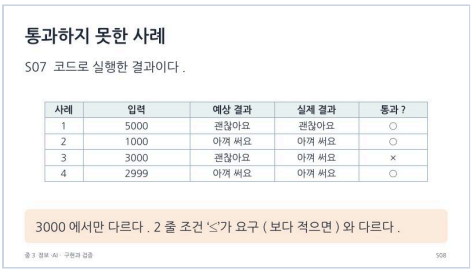
- 사례 1 (5000): 예상 괜찮아요, 실제 괜찮아요, ○
- 사례 2 (1000): 예상 아껴 써요, 실제 아껴 써요, ○
- 사례 3 (3000): 예상 괜찮아요, 실제 아껴 써요, ×
- 사례 4 (2999): 예상 아껴 써요, 실제 아껴 써요, ○
- ③ 사례 3(3000)이 통과하지 못해요.

막히면

- 예상 결과를 쓸 때 S06의 요구 문장을 소리 내어 읽고 "3000은 3000보다 적어?"만 묻습니다.

주의

- 5000과 1000만 넣었다면 이 오류를 찾지 못했습니다. 경계 값 사례의 힘을 아이가 스스로 느끼게 합니다.



S08 통과하지 못한 사례 · 통과하지 못한 사례가 알려 주는 오류의 위치 · 약 3분

통과하지 못한 사례(3000)를 실마리로 그 입력으로 코드를 따라가며 예상과 달라지는 줄(2줄의 ≤)을 찾는 시범을 보입니다. 이렇게 오류를 찾아 고치는 것을 디버깅이라고 합니다.

이렇게 말해 보세요

- "어느 사례가 ×야? 그 입력으로 코드를 다시 따라가면 어느 줄에서 예상과 달라져?"
- "요구는 '보다 적으면'이었어. 코드에는 무엇이 쓰여 있어?"

예상 답

- "2줄이요. ≤라서 3000도 '아껴 써요'가 나와요."

정답

- 사례 3(3000): 예상 '괜찮아요', 실제 '아껴 써요' → 2줄 '남은돈 ≤ 3000'을 '남은돈 < 3000'으로 고쳐야 해요.

막히면

- S16(보충)의 '3000 < 3000', '3000 ≤ 3000' 판단을 함께 해 봅니다.

주의

- 디버깅(debugging)은 '벌레(bug) 잡기'라는 뜻에서 나온 말로, 프로그램의 오류를 찾아 고치는 일입니다.



S09 **어떻게 고칠까?** · 오개념 비교(특정 사례만 맞추는 고치기 vs 원인 고치고 모두 다시 검증) · 약 2분

오류는 결과만 맞추지 않고 원인(조건)을 고쳐야 하며, 고친 뒤에는 앞서 통과한 사례까지 모두 다시 검증해야 한다는 것을 확인합니다(M4).

이렇게 말해 보세요

- "민재처럼 고치면 기준이 4000원으로 바뀌었을 때도 맞을까?"
- "고치다가 다른 곳이 망가질 수도 있어. 그래서 무엇을 다시 해야 해?"

예상 답

- "유나요. 원인을 고쳐야 하고, 다른 사례가 망가지지 않았는지 다 다시 봐야 해요."

정답

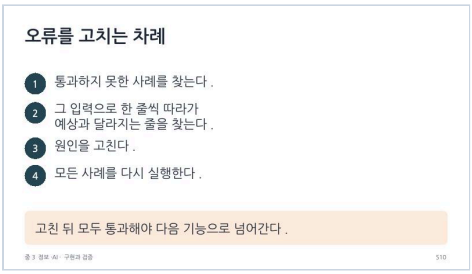
- 유나의 방법이 낫습니다. 오류의 원인인 조건을 요구에 맞게 고치고, 모든 사례를 다시 실행해 통과를 확인합니다. 민재처럼 한 값만 따로 막으면 코드가 복잡해지고, 원인(≤)이 남아 기준이 바뀌면 또 틀립니다.

막히면

- 민재의 고친 코드를 말로 따라가며 줄이 몇 개로 늘어나는지 세어 봅니다.

주의

- 지치면 여기서 쉬고, 다음 날 S10부터 이어도 됩니다.



S10 **오류를 고치는 차례** · 오류를 찾아 고치는 차례(디버깅) · 약 2분

S08·S09에서 한 일을 디버깅 차례 네 단계로 정리합니다.

이렇게 말해 보세요

- "S08에서 우리가 한 일을 네 단계로 말해 볼래?"
- "4단계를 건너뛰면 어떤 일이 생길 수 있을까?"

예상 답

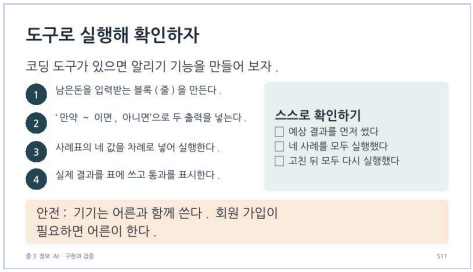
- "고치다가 다른 사례가 틀리게 될 수 있어요."

막히면

- S08 표를 다시 보며 단계마다 무엇을 했는지 짚어 줍니다.

주의

- 고친 코드 '남은돈 < 3000'으로 네 사례를 다시 실행하면 모두 통과합니다(3001을 더 넣어도 '괜찮아요').



S11 도구로 실행해 확인하자 · 도구로 구현하고 사례 실행(선택) · 약 3분

쓸 수 있는 도구(엔트리·스크래치 같은 블록 코딩 도구, 또는 파이썬)로 알리기 기능을 구현하고 사례표의 입력을 넣어 실행해 봅니다. 컴퓨터가 없으면 부모가 '컴퓨터 역할'을 맡아 아이가 불러 주는 코드를 한 줄씩 그대로 실행합니다.

이렇게 말해 보세요

- "먼저 ≤로 만들어 3000을 넣어 볼래? 무엇이 나와?"
- "<로 고친 뒤 네 사례를 모두 다시 넣어 보자."

예상 답

- "≤일 때 3000을 넣으면 아껴 써요가 나와요. 고치니까 괜찮아요가 나와요."

정답

- 고친 코드로 5000 → 괜찮아요, 1000 → 아껴 써요, 3000 → 괜찮아요, 2999 → 아껴 써요가 나오면 됩니다.

막히면

- 도구가 없거나 어렵다면 종이로 하고, 부모가 컴퓨터 역할을 합니다.

주의

- 블록 도구는 '묻고 기다리기' 같은 블록으로 입력을, '말하기' 블록으로 출력을 합니다. 파이썬 같은 글 코딩 도구에서는 if·else로 씁니다. 블록 이름과 위치는 도구와 판(버전)마다 다릅니다.
- 입력받은 값이 글자로 들어오면 수로 바꿔야 비교가 바르게 됩니다(파이썬은 int(…)로 바꿉니다).
- 이 장은 선택입니다. 기기가 없으면 건너뛰어도 목표 확인에는 지장이 없습니다.

안전: 기기는 어른과 함께 쓴다. 회원 가입이 필요하면 어른이 한다.



S12 **혼자 해 보기** · 혼자 해 보기(사흘 합계 기능 검증과 디버깅) · 약 5분

처음 보는 반복 코드에 테스트 사례를 적용해 예상과 실제 결과를 견주고(A), 통과하지 못한 까닭을 따라가 찾아 고친 뒤 다시 검증할 사례를 만드는지(B) 확인합니다(활동지 3).

이렇게 말해 보세요

- “이번에는 엄마(아빠)가 도와주지 않을게. 예상 결과부터 먼저 써 줘.”
- “코드를 따라가면 쓴돈을 몇 번 입력받아?”

정답

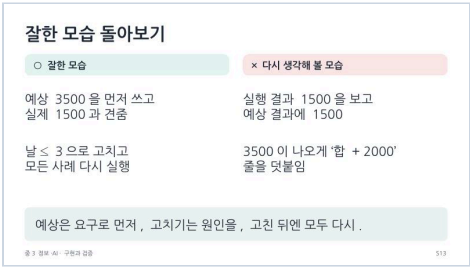
- ① 예상 3500, 실제 1500 (날이 1, 2일 때만 반복해 두 번만 입력받고 더함) → 통과 못 함
- ② 3줄 ‘날 < 3’을 ‘날 ≤ 3’(또는 ‘날 < 4’)으로 고쳐요.
- ③ 예: 0, 0, 700 → 예상 700 (오류 코드라면 0이 나와 오류가 드러나는 사례). 셋째 날에 0이 아닌 값을 넣고 예상 결과를 맞게 쓰면 맞습니다.
- ①에서 예상을 먼저 쓰고 실제 결과를 바르게 따라가면 목표 A, ②에서 원인을 고치고 ③에서 다시 검증할 사례를 만들면 목표 B 확인 완료입니다.

막히면

- ②에서 막히면 “날이 3일 때 반복 안으로 들어가?”만 묻습니다. 그 이상 도우면 기록표에 ‘도움’으로 표시합니다.

주의

- 입력을 1000, 500, 0으로 하면 오류 코드도 1500이 나와 통과해 버립니다. 셋째 날 값이 0이면 오류가 숨는다는 것을 알아차리면 크게 칭찬합니다.
- 관찰한 것을 기록표(S12 줄)에 바로 표시합니다.



S13 학생용에서 빠짐 **잘한 모습 돌아보기** · 잘한 모습과 다시 생각해 볼 모습 · 약 2분

S12에서 아이가 쓴 사례표와 고친 코드를 잘한 모습, 다시 생각해 볼 모습과 견주며 돌아봅니다.

이렇게 말해 보세요

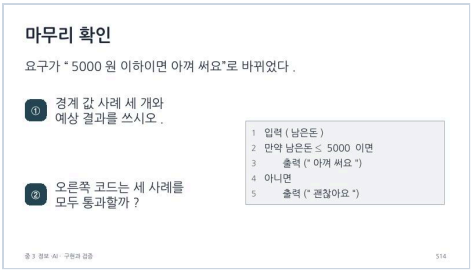
- “예상 결과는 언제 썼어?”
- “입력이 바뀌어도 네가 고친 코드가 맞을까? 0, 0, 700을 넣어 볼래?”

예상 답

- “코드 보기 전에 썼어요.” / “700이 나와요.”

주의

- 이 장은 학생용 파일에서 빠집니다. 잘한 점을 먼저 말하고, 고칠 점은 하나만 알려 줍니다.



S14 마무리 확인 · 마무리 확인(바뀐 요구에 맞는 경계 값 사례 만들기) · 약 3분

바뀐 요구에서 경계 값 사례를 스스로 골라 예상 결과를 정하고, 코드를 따라가 통과 여부를 판단하는지 마무리로 확인합니다(목표 A).

이렇게 말해 보세요

- "'이하'에는 5000이 들어갈까?"
- "결과가 바뀌는 곳 근처의 값 세 개를 골라 볼래?"

정답

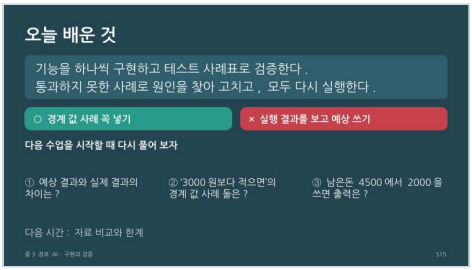
- ① 4999 → 아껴 써요, 5000 → 아껴 써요, 5001 → 괜찮아요 (5000과 그 양옆 값을 고르고 예상 결과가 맞으면 맞음)
- ② 모두 통과해요. '이하'는 ≤이므로 코드가 요구와 같아요(4999 → 아껴 써요, 5000 → 아껴 써요, 5001 → 괜찮아요).
- 경계 값 셋과 예상 결과를 바르게 쓰고 통과 여부를 까닭과 함께 말하면 확인 완료입니다.

막히면

- S06 카드를 다시 보여 줍니다. 기록표에 '도움'으로 표시합니다.

주의

- '이하'는 ≤, '미만·보다 적으면'은 < 입니다. 같은 기능이라도 요구의 낱말에 따라 맞는 코드가 달라집니다.



S15 오늘 배운 것 · 정리와 다음 학습 · 약 2분

오늘 배운 것을 정리하고 다음 수업 첫 5분에 할 복습을 알려 줍니다.

이렇게 말해 보세요

- "오늘 오류를 찾게 해 준 사례는 어떤 사례였지?"
- "다음 수업을 시작할 때 이 세 문제를 먼저 풀어 보자."

예상 답

- "3000이요. 경계 값이에요."

정답

- ① 예상 결과: 실행 전에 요구로 정한 결과, 실제 결과: 실행해서 나온 결과 ② 3000과 2999(3001도 됨) ③ 2500

주의

- 다음 차시는 '자료 비교와 한계'입니다. 같은 질문에 대한 두 조사 자료를 견주고 결론의 한계를 따집니다. 용돈 기록기 자료(구조도·코드)는 '프로젝트 공유 점검'에서 다시 씁니다.



S16 **보충: 참일까, 거짓일까?** · 보충(조건의 참·거짓 판단) · 약 4분

S02나 S08에서 < 와 ≤ 의 판단을 어려워하면 수직선으로 다시 확인합니다.

이렇게 말해 보세요

- "3000은 3000보다 작아? 그럼 3000 < 3000은?"
- "≤ 는 작거나 같으면 참이야. 3000 ≤ 3000은?"

정답

- ① ○ ② × ③ ○ ④ ×

막히면

- 이 차시의 선수 차시는 '문제 분해와 설계'입니다. 보충으로도 어려우면 그 차시의 '기능 하나를 순서도로'(S07)와 '알리는 기능에는 판단이'(S08)를 먼저 함께 본 뒤 S03부터 다시 합니다.

주의

- 보충이 오래 걸리면 오늘은 여기까지 하고, 다음 날 S02부터 다시 시작합니다.

활동지 정답

번호	정답
1	사례 1: 예상 3500, 실제 3500, ○ / 사례 2: 예상 0, 실제 0, ○ / 사례 3: 예상 3000, 실제 3000, ○ / 사례 4 예: 남은돈 0, 쓴돈 0 → 0, 0, ○. 예상 결과를 계산으로 맞게 쓰면 맞음.
2	5000: 예상 괜찮아요, 실제 괜찮아요, ○ / 1000: 예상 아껴 써요, 실제 아껴 써요, ○ / 3000: 예상 괜찮아요, 실제 아껴 써요, × / 2999: 예상 아껴 써요, 실제 아껴 써요, ○ → 2줄 '남은돈 ≤ 3000'을 '남은돈 < 3000'으로
3	사례 1: 예상 3500, 실제 1500, × / 사례 2(아이가 정함) 예: 0, 0, 700 → 예상 700, 실제 0, × / 3줄 '날 < 3'을 '날 ≤ 3'으로. 고친 뒤 두 사례 모두 통과.
4	4999 → 아껴 써요, 5000 → 아껴 써요, 5001 → 괜찮아요 (5000과 양옆 값, 이하는 5000 포함)
5	예: 입력(남은돈), 입력(쓴돈), 만약 쓴돈 > 남은돈 이면 출력("돈이 모자라요") 아니면 남은돈 ← 남은돈 − 쓴돈, 출력(남은돈). 사례 예: (3000, 1000) → 2000, (3000, 3000) → 0(경계 값: 같을 때는 모자라지 않음), (3000, 3500) → 돈이 모자라요. 예상 결과를 먼저 쓰고 경계(같을 때)를 넣었으면 맞음.
6	① 예상 결과는 실행 전에 요구로 정하고, 실제 결과는 실행해서 나온 것 ② 3000(괜찮아요), 2999(아껴 써요) — 3001도 맞음 ③ 2500

오늘의 기록

S07 알리기 기능 검증: 예상 먼저·3000에서 × 찾음 □ 일부 □ 도움을 받아 □
S12 혼자 해 보기: ① 예상·실제 □ ② 고치기 □ ③ 새 사례 □ 도움 □
S14 마무리 확인: 경계 값 셋과 통과 판단 □ 일부 □ 아직 □
S11 도구로 실행(선택): 했음 □ 부모가 컴퓨터 역할 □ 건너뜀 □

메모: _____